

GUIA DE ESTUDO

Banco de Dados do Sistema de Controle de Estoque

Modelagem, SQLite, SQL, integridade, migração e perguntas da banca

Equipe 02 — Edypho, Lucas e Nicollas

1. O que você precisa saber primeiro

O projeto usa SQLite, um banco relacional embutido no Python. Os dados ficam no arquivo estoque.db e são acessados pelo módulo sqlite3.

- Três tabelas: categorias, produtos e movimentacoes.
- Uma categoria possui vários produtos; um produto possui várias movimentações.
- Chaves primárias: id. Chaves estrangeiras: categoria_id e produto_id.
- Entrada e saída atualizam o saldo e gravam histórico na mesma transação.
- Produto removido recebe ativo = 0; não é apagado.
- Migrações adicionam colunas sem destruir dados existentes.

Resposta de abertura: “Nosso banco é SQLite e foi modelado de forma relacional. Categorias organizam produtos e produtos geram movimentações. Usamos chaves estrangeiras, comandos parametrizados, transações e exclusão lógica para preservar a integridade e o histórico.”

2. Diagrama e cardinalidade

CATEGORIAS	1 : N	PRODUTOS	1 : N	MOVIMENTACOES
id (PK) nome UNIQUE ativo	categoria_id (FK)	id (PK) quantidade estoque_minimo ativo	produto_id (FK)	id (PK) tipo quantidade saldos

Cardinalidade 1:N: uma linha da tabela pai pode se relacionar com várias linhas da tabela filha. Cada produto pertence a apenas uma categoria e cada movimentação pertence a apenas um produto.

3. Estado do banco no momento deste guia

Item	Valor confirmado
Arquivo	estoque.db na raiz do projeto
Tamanho	32 KB no momento da geração
Categorias	9 registros
Produtos	26 registros
Movimentações	52 registros
SQLite da execução consultada	3.40.1

Os números mudam conforme o sistema é utilizado. O importante é saber consultar com SELECT COUNT(*)

4. Dicionário de dados — categorias

Campo	Tipo	Regra	Finalidade
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Identificador único.
nome	TEXT	NOT NULL, UNIQUE	Nome da categoria.
descricao	TEXT	Opcional	Detalhes da categoria.
ativo	INTEGER	NOT NULL DEFAULT 1	1 = ativa; 0 = inativa.
criado_em	TEXT	CURRENT_TIMESTAMP em banco novo	Data de criação.

O UNIQUE de nome cria automaticamente um índice interno no SQLite. No banco analisado ele aparece como `sqlite_autoindex_categorias_1`.

A API lista apenas categorias com `ativo = 1`. Ainda não existe rota de inativação de categoria.

5. Dicionário de dados — produtos

Campo	Tipo	Regra / uso
id	INTEGER	Chave primária autoincremental.
nome	TEXT	Obrigatório.
categoria_id	INTEGER	FK para categorias.id; obrigatório.
preco	REAL	Obrigatório e validado como não negativo.
quantidade	INTEGER	Saldo atual; padrão 0.
estoque_minimo	INTEGER	Limite lógico para alerta; padrão 0.
descricao	TEXT	Opcional; ainda não exposto no cadastro da API.
sku	TEXT	Opcional; utilizado pelo seed, não pela tela atual.
fornecedor	TEXT	Opcional; ainda não exposto na API.
ativo	INTEGER	Exclusão lógica: 1 ativo, 0 inativo.
criado_em	TEXT	Data de criação em bancos novos.

6. Dicionário de dados — movimentacoes

Campo	Tipo	Finalidade
id	INTEGER	Chave primária.
produto_id	INTEGER	FK para produtos.id.
tipo	TEXT	ENTRADA ou SAIDA.
quantidade	INTEGER	Unidades movimentadas.
observacao	TEXT	Motivo ou descrição opcional.
data_hora	TEXT	Data/hora no formato YYYY-MM-DD HH:MM:SS.
saldo_anterior	INTEGER	Saldo antes da operação.
saldo_atual	INTEGER	Saldo depois da operação.

7. Restrições e integridade

Mecanismo	O que garante	Observação
PRIMARY KEY	ID único por tabela.	INTEGER PRIMARY KEY usa o rowid do SQLite.
AUTOINCREMENT	Não reutiliza IDs já emitidos.	Gera o próximo identificador.
NOT NULL	Campos obrigatórios não recebem NULL.	Não impede string vazia; isso é validado no Python.
UNIQUE	Nome de categoria não se repete.	IntegrityError vira CategoriaDuplicadaError.
FOREIGN KEY	Produto referencia categoria e movimento referencia produto.	Exige PRAGMA foreign_keys = ON em cada conexão.
DEFAULT	Fornece valor quando campo é omitido.	Ex.: quantidade 0 e ativo 1.

Importante: regras como quantidade positiva, tipo ENTRADA/SAIDA e preço não negativo estão principalmente no Python. O esquema não possui CHECK para todas elas.

Melhoria possível: adicionar CHECK (quantidade >= 0), CHECK (estoque_minimo >= 0), CHECK (tipo IN ('ENTRADA','SAIDA')) e índices nas FKs.

8. SQL parametrizado e prevenção de injeção

```
conn.execute(
    "SELECT * FROM produtos WHERE id = ? AND ativo = 1",
    (produto_id,)
)
```

O ponto de interrogação é um placeholder. O driver envia o valor separadamente, sem tratá-lo como parte do comando SQL.

Na migração há f-string para nomes de tabela/coluna. Nesse caso os nomes vêm de constantes internas do código, nunca da entrada do usuário.

9. Transações e atomicidade

- Consulta o produto e lê saldo_anterior.
- Valida se uma saída pode acontecer.
- Insere a linha em movimentacoes.
- Atualiza produtos.quantidade.
- commit confirma as duas mudanças.
- rollback desfaz tudo se qualquer etapa falhar.

Isso é atomicidade: não pode existir histórico sem saldo atualizado, nem saldo atualizado sem histórico.

10. Exclusão lógica

```
UPDATE produtos SET ativo = 0 WHERE id = ?
```

O produto deixa de aparecer porque as listagens usam WHERE ativo = 1. As movimentações continuam referenciando o mesmo produto e o relatório geral preserva o histórico.

A chave estrangeira não usa ON DELETE CASCADE. Isso é coerente com a decisão de não apagar produtos fisicamente.

11. Criação e migração do banco

- CREATE TABLE IF NOT EXISTS cria somente as tabelas ausentes.
- PRAGMA table_info(tabela) lista as colunas existentes.
- adicionar_coluna_se_nao_existir executa ALTER TABLE apenas quando necessário.
- A migração preserva dados antigos e é coberta por test_database.py.

Detalhe real: em banco novo, criado_em é NOT NULL DEFAULT CURRENT_TIMESTAMP. Em banco antigo migrado, a coluna é adicionada como TEXT opcional, pois o SQLite não permite todas as mudanças de restrição de forma simples com ALTER TABLE. Isso preserva compatibilidade, mas pode deixar criado_em nulo em registros antigos.

12. Consultas usadas pelo sistema

Necessidade	Ideia da consulta
Listar produtos	SELECT ... FROM produtos WHERE ativo = 1 ORDER BY nome
Estoque baixo	Lista ativos e filtra quantidade <= estoque_minimo no Python
Inventário	Quantidade × preço para gerar valor_total
Histórico	WHERE produto_id = ? ORDER BY data_hora DESC, id DESC
Dashboard	COUNT(*) por tipo ENTRADA ou SAIDA
Categoria válida	SELECT id WHERE id = ? AND ativo = 1

O dashboard conta operações, não soma unidades. Para total de unidades seria SUM(quantidade).

13. Seed e testes do banco

backend/seed.py cria categorias, produtos e 50 movimentações de informática para demonstração. Produtos são encontrados pelo SKU e as observações recebem uma tag.

- Se a tag já existir, o seed não cria novamente suas movimentações.
- test_database.py monta manualmente um esquema antigo e executa criar_tabelas().
- O teste confirma categoria preservada e colunas sku, ativo, saldo_anterior e saldo_atual.
- Outros testes usam bancos separados e os removem ao terminar.

14. Limitações e melhorias

Situação atual	Melhoria possível
Preço REAL	Centavos em INTEGER ou Decimal para precisão monetária.
Poucos índices	Índices em categoria_id, produto_id, ativo e data_hora.
Regras no Python	CHECK constraints também no banco.
SQLite local	PostgreSQL/MySQL para mais concorrência e múltiplos servidores.
Sem backup automático	Rotina de cópia, restauração e versionamento.
Migração manual simples	Ferramenta de migração com versões, como Alembic em arquitetura maior.
Data/hora TEXT local	UTC e formato ISO 8601 com timezone.

15. Perguntas fundamentais de banco de dados

Treine a resposta curta primeiro. Se a banca pedir detalhes, mostre a tabela, o SQL ou o fluxo da transação.

PERGUNTA	O que é um banco de dados?
RESPOSTA	Conjunto organizado de dados que permite armazenar, consultar e atualizar informações.
PERGUNTA	O que é um banco relacional?
RESPOSTA	Organiza dados em tabelas relacionadas por chaves.
PERGUNTA	O que é uma tabela?
RESPOSTA	Estrutura formada por colunas e linhas que representa uma entidade ou evento.
PERGUNTA	O que é registro?
RESPOSTA	Uma linha da tabela; por exemplo, um produto específico.
PERGUNTA	O que é coluna?
RESPOSTA	Um atributo do registro, como nome, preço ou quantidade.
PERGUNTA	Quais tabelas o projeto possui?
RESPOSTA	categorias, produtos e movimentacoes.
PERGUNTA	O que é chave primária?
RESPOSTA	Campo que identifica unicamente cada linha; usamos id.
PERGUNTA	O que é chave estrangeira?
RESPOSTA	Campo que referencia a chave primária de outra tabela.
PERGUNTA	Quais são as FKs?
RESPOSTA	produtos.categoria_id e movimentacoes.produto_id.
PERGUNTA	Qual é a cardinalidade?
RESPOSTA	Categoria 1:N produtos e produto 1:N movimentações.
PERGUNTA	O que significa NULL?
RESPOSTA	Ausência de valor; diferente de zero ou texto vazio.
PERGUNTA	O que é integridade referencial?
RESPOSTA	Garantia de que uma FK aponta para um registro válido.

16. Perguntas sobre SQLite

Treine a resposta curta primeiro. Se a banca pedir detalhes, mostre a tabela, o SQL ou o fluxo da transação.

PERGUNTA	Por que SQLite?
RESPOSTA	É leve, local, gratuito, não exige servidor e atende ao projeto acadêmico.
PERGUNTA	Onde o banco fica?
RESPOSTA	No arquivo estoque.db na raiz.
PERGUNTA	SQLite é SQL?
RESPOSTA	SQLite é um SGBD que entende a linguagem SQL.
PERGUNTA	O que é SGBD?
RESPOSTA	Software que gerencia armazenamento, consultas, transações e integridade do banco.
PERGUNTA	SQLite aceita vários usuários?
RESPOSTA	Aceita acessos, mas tem limitações de escritas concorrentes comparado a bancos cliente-servidor.
PERGUNTA	Para que serve sqlite3 no Python?
RESPOSTA	É o módulo usado para abrir conexões e executar SQL no SQLite.
PERGUNTA	Para que serve row_factory?
RESPOSTA	sqlite3.Row permite acessar os resultados pelo nome da coluna.
PERGUNTA	Por que ativar foreign_keys?
RESPOSTA	No SQLite essa fiscalização precisa ser habilitada por conexão.
PERGUNTA	O que é PRAGMA?
RESPOSTA	Comando específico do SQLite para consultar/configurar seu comportamento.
PERGUNTA	O que faz PRAGMA table_info?
RESPOSTA	Retorna nomes, tipos, padrões, NOT NULL e PK das colunas.
PERGUNTA	O que acontece se estoque.db não existir?
RESPOSTA	A conexão cria o arquivo e criar_tabelas cria as estruturas.
PERGUNTA	SQLite seria ideal para produção grande?
RESPOSTA	Não necessariamente; alta escala pode exigir PostgreSQL ou MySQL.

17. Perguntas sobre o esquema do projeto

Treine a resposta curta primeiro. Se a banca pedir detalhes, mostre a tabela, o SQL ou o fluxo da transação.

PERGUNTA	Por que separar categorias?
RESPOSTA	Evita repetir descrição de categoria em cada produto e organiza os dados.
PERGUNTA	Por que movimentações têm tabela própria?
RESPOSTA	Para manter histórico de cada evento sem sobrescrever informações anteriores.
PERGUNTA	Por que guardar quantidade também em produtos?
RESPOSTA	Permite consultar rapidamente o saldo atual sem somar todo o histórico.
PERGUNTA	Isso duplica informação?
RESPOSTA	Há saldo atual e histórico, mas a transação mantém ambos consistentes; é uma decisão por desempenho e simplicidade.
PERGUNTA	Por que saldo_anterior e saldo_atual?
RESPOSTA	Auditoria e rastreabilidade de cada alteração.
PERGUNTA	Por que ativo é INTEGER?
RESPOSTA	SQLite não tem BOOLEAN rígido; 1 representa verdadeiro e 0 falso.
PERGUNTA	Por que data_hora é TEXT?
RESPOSTA	SQLite não possui tipo DATETIME rígido; texto ordenável no padrão usado atende ao projeto.
PERGUNTA	Por que nome de categoria é UNIQUE?
RESPOSTA	Para impedir duplicidade lógica.
PERGUNTA	SKU é único?
RESPOSTA	Não há UNIQUE no esquema atual. O seed o usa para localizar produto, mas uma melhoria seria impor unicidade quando preenchido.
PERGUNTA	Por que não existe ON DELETE CASCADE?
RESPOSTA	Porque o produto não é apagado; ele é inativado para preservar o histórico.
PERGUNTA	Campos opcionais podem ser NULL?
RESPOSTA	Sim, como descricao, sku e fornecedor.
PERGUNTA	criado_em é sempre preenchido?
RESPOSTA	Em bancos novos sim por padrão; registros antigos migrados podem ficar nulos.

18. Perguntas sobre SQL, transações e segurança

Treine a resposta curta primeiro. Se a banca pedir detalhes, mostre a tabela, o SQL ou o fluxo da transação.

PERGUNTA	Diferença entre SELECT, INSERT e UPDATE?
RESPOSTA	SELECT consulta, INSERT cria linha e UPDATE altera linha existente.
PERGUNTA	O DELETE da API executa DELETE SQL?
RESPOSTA	Não. Executa UPDATE ativo = 0, uma exclusão lógica.
PERGUNTA	O que é WHERE?
RESPOSTA	Condição que escolhe quais linhas serão consultadas ou alteradas.
PERGUNTA	O que é ORDER BY?
RESPOSTA	Ordena os resultados; DESC coloca os maiores/mais recentes primeiro.
PERGUNTA	COUNT e SUM são iguais?
RESPOSTA	Não. COUNT conta linhas/operações; SUM soma valores, como unidades.
PERGUNTA	O que é transação?
RESPOSTA	Conjunto de operações confirmado ou desfeito como uma unidade.
PERGUNTA	O que é commit?
RESPOSTA	Confirma as alterações da transação.
PERGUNTA	O que é rollback?
RESPOSTA	Desfaz alterações não confirmadas após erro.
PERGUNTA	O que é atomicidade?
RESPOSTA	Todas as operações da unidade funcionam ou nenhuma fica gravada.
PERGUNTA	Como evitam SQL Injection?
RESPOSTA	Usam placeholders ? para valores do usuário.
PERGUNTA	Por que fechar a conexão?
RESPOSTA	Libera arquivo, locks e recursos do sistema.
PERGUNTA	O que ocorre se o INSERT funcionar e UPDATE falhar?
RESPOSTA	A exceção executa rollback, desfazendo o INSERT.

19. Perguntas difíceis e respostas honestas

Treine a resposta curta primeiro. Se a banca pedir detalhes, mostre a tabela, o SQL ou o fluxo da transação.

PERGUNTA	O banco garante que quantidade nunca seja negativa?
RESPOSTA	A regra está no Python. O banco poderia ser reforçado com CHECK quantidade >= 0.
PERGUNTA	O banco garante que tipo seja ENTRADA/SAIDA?
RESPOSTA	Hoje o Python controla; um CHECK no esquema seria uma melhoria.
PERGUNTA	REAL é seguro para dinheiro?
RESPOSTA	Pode ter imprecisão binária. Melhor usar centavos INTEGER ou Decimal em sistemas financeiros.
PERGUNTA	Há índices suficientes?
RESPOSTA	Há índice automático para categoria.nome, mas FKs e filtros frequentes poderiam receber índices.
PERGUNTA	Pode ocorrer corrida com duas saídas simultâneas?
RESPOSTA	No uso local é pouco provável; produção exigiria transação/lock mais rígido ou UPDATE condicional em SGBD robusto.
PERGUNTA	A migração é completa?
RESPOSTA	É adequada ao escopo e adiciona colunas. Não controla versões nem altera restrições complexas.
PERGUNTA	Como fariam backup?
RESPOSTA	Copiaríamos o arquivo com aplicação parada ou usariam a API de backup do SQLite e testariam restauração.
PERGUNTA	E se o arquivo corromper?
RESPOSTA	Sem backup haveria perda; produção precisa política de backup, integridade e recuperação.
PERGUNTA	Por que SQL direto e não ORM?
RESPOSTA	SQL direto é transparente e educativo para iniciantes; ORM seria uma evolução possível.
PERGUNTA	Existe normalização?
RESPOSTA	Sim: categorias e movimentações estão separadas de produtos. O saldo atual é uma desnormalização controlada.
PERGUNTA	Qual a maior decisão de integridade?
RESPOSTA	Usar transação para histórico+saldo e exclusão lógica para preservar relacionamentos.
PERGUNTA	O que melhorariam primeiro?
RESPOSTA	CHECKs, índices, precisão monetária, migrações versionadas e estratégia de backup.

20. Resumo de bolso — banco de dados

Tema	Resposta rápida
SGBD	SQLite, relacional e armazenado em estoque.db.
Tabelas	categorias, produtos e movimentacoes.
Relacionamentos	categoria 1:N produto; produto 1:N movimentação.
PK / FK	id; categoria_id e produto_id.
Saldo	Guardado em produtos e auditado em movimentacoes.
Transação	INSERT do histórico + UPDATE do saldo + commit/rollback.
Remoção	UPDATE ativo = 0, preservando histórico.
Segurança SQL	Placeholders ? para valores externos.
Migração	PRAGMA table_info + ALTER TABLE sem apagar dados.
Limitações	REAL para preço, poucos índices/CHECKs e concorrência do SQLite.

Fechamento recomendado: “A modelagem atende ao escopo do estoque e prioriza integridade do histórico. Para uma versão de produção, evoluiríamos restrições, índices, backup, migrações e o SGBD conforme a escala.”